

Week 3

1.

```
1 class Solution {
2     public long findMinDiff (ArrayList<Integer> arr, int m) {
3         int n = arr.size();
4         if (m > n)
5             return -1;
6         Collections.sort(arr);
7         long minDiff = Long.MAX_VALUE;
8         for (int i = 0; i + m - 1 < n; i++) {
9             long diff = arr.get(i + m - 1) - arr.get(i);
10            minDiff = Math.min(minDiff, diff);
11        }
12        return minDiff;
13    }
14 }
```

1.

```
1 class Solution {
2     public static int smallestSubWithSum(int x, int[] arr) {
3         int n = arr.length;
4         int minLen = n + 1;
5         int start = 0, sum = 0;
6         for (int end = 0; end < n; end++) {
7             sum += arr[end];
8             while (sum > x) {
9                 minLen = Math.min(minLen, end - start + 1);
10                sum -= arr[start];
11                start++;
12            }
13        }
14        if (minLen == n + 1)
15            return 0;
16        return minLen;
17    }
18    public static void main(String[] args) {
19        int[] arr = {1, 4, 45, 6, 0, 19};
20        int x = 51;
21        System.out.println(smallestSubWithSum(x, arr));
22    }
23 }
```

1.

```
1 class Solution {
2     public static boolean threeWayPartition(int[] arr, int a, int b) {
3         int low = 0, mid = 0;
4         int high = arr.length - 1;
5         while (mid <= high) {
6             if (arr[mid] < a) {
7                 int temp = arr[low];
8                 arr[low] = arr[mid];
9                 arr[mid] = temp;
10                low++;
11                mid++;
12            }
13            else if (arr[mid] > b) {
14                int temp = arr[mid];
15                arr[mid] = arr[high];
16                arr[high] = temp;
17                high--;
18            }
19            else {
20                mid++;
21            }
22        }
23        return true;
24    }
25 }
```

1

```
1 class Solution {
2     public static int minSwap(int[] arr, int k) {
3         int n = arr.length;
4         int good = 0;
5         for (int i = 0; i < n; i++) {
6             if (arr[i] <= k)
7                 good++;
8         }
9         if (good == 0)
10            return 0;
11        int bad = 0;
12        for (int i = 0; i < good; i++) {
13            if (arr[i] > k)
14                bad++;
15        }
16        int ans = bad;
17        int i = 0, j = good;
18        while (j < n) {
19            if (arr[i] > k)
20                bad--;
21            if (arr[j] > k)
22                bad++;
23            ans = Math.min(ans, bad);
24            i++;
25            j++;
26        }
27        return ans;
28    }
29 }
```

2.

```
1 import java.util.*;
2 class Solution {
3     public static double findMedian(int[] arr) {
4         int n = arr.length;
5         Arrays.sort(arr);
6         if (n % 2 == 1) {
7             return arr[n / 2];
8         }
9         else {
10            return (arr[n / 2 - 1] + arr[n / 2]) / 2.0;
11        }
12    }
13 }
```

2.

```
1 import java.util.*;
2 class Solution {
3     public static ArrayList<Integer> spirallyTraverse(int[][] mat) {
4         ArrayList<Integer> result = new ArrayList<>();
5         int n = mat.length;
6         int m = mat[0].length;
7         int top = 0, bottom = n - 1;
8         int left = 0, right = m - 1;
9         while (top <= bottom && left <= right) {
10             for (int i = left; i <= right; i++) {
11                 result.add(mat[top][i]);
12             }
13             top++;
14             for (int i = top; i <= bottom; i++) {
15                 result.add(mat[i][right]);
16             }
17             right--;
18             if (top <= bottom) {
19                 for (int i = right; i >= left; i--) {
20                     result.add(mat[bottom][i]);
21                 }
22                 bottom--;
23             }
24             if (left <= right) {
25                 for (int i = bottom; i >= top; i--) {
26                     result.add(mat[i][left]);
27                 }
28                 left++;
29             }
30         }
31         return result;
32     }
33 }
```

2.

```
1  class Solution {  
2      public boolean searchMatrix(int[][] matrix, int target) {  
3  
4          int m = matrix.length;  
5          int n = matrix[0].length;  
6  
7          int low = 0;  
8          int high = m * n - 1;  
9  
10         while (low <= high) {  
11             int mid = low + (high - low) / 2;  
12  
13             int row = mid / n;  
14             int col = mid % n;  
15  
16             if (matrix[row][col] == target)  
17                 return true;  
18             else if (matrix[row][col] < target)  
19                 low = mid + 1;  
20             else  
21                 high = mid - 1;  
22         }  
23  
24         return false;  
25     }  
26 }
```

2.

```
1 class Solution {
2     public static int median(int[][] mat) {
3         int n = mat.length;
4         int m = mat[0].length;
5         int low = 1, high = 2000;
6         int desired = (n * m) / 2;
7         while (low <= high) {
8             int mid = (low + high) / 2;
9             int count = 0;
10            for (int i = 0; i < n; i++) {
11                count += upperBound(mat[i], mid);
12            }
13            if (count <= desired)
14                low = mid + 1;
15            else
16                high = mid - 1;
17        }
18        return low;
19    }
20    static int upperBound(int[] row, int target) {
21        int l = 0, r = row.length;
22        while (l < r) {
23            int mid = (l + r) / 2;
24            if (row[mid] <= target)
25                l = mid + 1;
26            else
27                r = mid;
28        }
29        return l;
30    }
31 }
```

2.

```
1 class Solution {
2     public static int rowWithMax1s(int[][] arr) {
3         int n = arr.length;
4         int m = arr[0].length;
5         int rowIndex = -1;
6         int j = m - 1;
7         for (int i = 0; i < n; i++) {
8             while (j >= 0 && arr[i][j] == 1) {
9                 j--;
10                rowIndex = i;
11            }
12        }
13        return rowIndex;
14    }
15 }
```